

Módulo 9

Projeto CRUD Completo + Tratamento de Erros

JORNADA DEV START · Programa START — TOTVS Paulista

Aula 9 · 29/07 · ao vivo

Você está aqui

```
1 → 2 → 3 → 4 → 5 → 6 | 7 → 8 → [9] → 10
Fundamentos (Harbour) | Protheus (ADVPL)
                        ▲ você está aqui
```

Reta final do Protheus: juntar tudo num **projeto de verdade** e blindá-lo. 

Roteiro de hoje (2h)

Bloco	Tempo
Retomada do CRUD da Aula 8	~10 min
Projeto 2 tabelas, lib, validações e erros	~60 min
Mão na massa — gravação à prova de falhas	~40 min
Dúvidas / networking	~10 min

Ao fim desta aula você será capaz de

- Modelar **duas tabelas relacionadas** (1:N) no dicionário
- Criar uma **biblioteca** de funções comuns (DRY)
- Ligar tabelas com **campos virtuais**, **gatilhos** e um **botão**
- Proteger a **integridade referencial**
- Capturar o inesperado com **BEGIN SEQUENCE** + transações + log

O projeto: Contatos e Interações

SZ1 – Contatos (1)

Z1_CODIGO (PK)
Z1_CLIENTE
Z1_ASSUNTO

—1:N—|

SZ2 – Interações (N)

Z2_CONTAT (FK→SZ1) |
Z2_TIPO / Z2_DESCRI |
Z2_DATA / Z2_USUAR |

Um contato tem **várias** interações → relação **1:N**.

Aprender UM ensina TODOS

 É a agenda do vendedor: **um** cliente com **várias** anotações de conversa. Cada anotação sabe de quem é; o cliente não precisa listar nada.

pedido → itens · cliente → endereços · nota → parcelas

Todos são **1:N** — o mesmo esqueleto de hoje.

? Checkpoint relâmpago

? Na relação Contato → Interações:

a) quem guarda a chave do outro? · b) um contato pode ter quantas interações?

Modelando os dados (dicionário)

Passo	Onde	O quê
Tabelas	SX2	SZ1 e SZ2 (Compartilhado)
Campos	SX3	tipo, tamanho, contexto
Índices	SIX	chaves de busca
Domínio	SX5	tipos de interação (E/L/R/V/W)

Índice-chave da SZ2: **Z2_FILIAL + Z2_CONTAT + Z2_SEQUEN**

Campos virtuais: aparecem sem ocupar espaço

```
// Z2_ASSUNT – assunto do contato, calculado na hora (X3_RELACA0):  
POSICIONE("SZ1", 1, xFilial("SZ1") + M->Z2_CONTAT, "Z1_ASSUNT0")
```

`POSICIONE(alias, ordem, chave, campo)` "pula" até o registro e devolve o campo.

Biblioteca de funções (não repita código!)

STTIPLIB.PRW — princípio **DRY** (*Don't Repeat Yourself*):

 É a receita da vó anotada UMA vez: todo mundo consulta a mesma folha. Corrigiu ali, corrigiu pra todos.

Regra mudou? Você altera **em um só lugar**.

STTIPLIB.PRW — funções da lib

```
STATIC FUNCTION NomeCliente(cCodigo, cLoja)
    RETURN AllTrim(POSICIONE("SA1", 1, ;
        xFilial("SA1") + cCodigo + cLoja, "A1_NOME"))

STATIC FUNCTION ProxCodigoSZ1()
    RETURN GetSXENum("SZ1", "Z1_CODIGO")
```

Cada função: **um nome claro, um propósito, um lugar só.**

Rotina SZ1 — botão + legendas

```
PRIVATE aRotina := {;
    {"Incluir",    "AxInclui",    0, 3},;
    {"Alterar",    "AxAlterar",    0, 4},;
    {"Interações", "U_STTIP004", 0, 6}; // ← customizado!
}
aColors := {;
    {"SZ1->Z1_DATA >= dDataBase - 7", "BR_GREEN"},;
    {".T.", "BR_RED"};
}
mBrowse(1, 1, 22, 75, "SZ1", , , , , , aColors, , , , , .F., , , cFiltro)
```

Anatomia da rotina SZ1

Elemento	O que faz
aRotina	opções do menu: {"Título","Função",0,0p}
{"Interações", "U_STTIP004", 0, 6}	opção customizada → nossa rotina
aColors	a 1ª regra que der .T. pinta a linha
mBrowse(...)	monta a grade com tudo configurado

Rotina SZ2 – filtrada pelo contato

```
USER FUNCTION STTIP004()  
  LOCAL cCodigo := SZ1->Z1_CODIGO      // contato atual  
  cFiltro := "Z2_CONTAT == '" + cCodigo + "'"  
  dbSelectArea("SZ2") ; dbSetOrder(1)  
  dbSeek(xFilial("SZ2") + cCodigo)  
  mBrowse(1, 1, 22, 75, "SZ2", , , , , , , , , , .F., , , cFiltro)  
  RETURN NIL
```

💡 Gatilhos (SX7) preenchem sozinhos: **Z2_DATA** = dDataBase · **Z2_HORA** = Time() · **Z2_USUAR** = cNomUsr. O usuário só digita tipo e descrição.

! Integridade referencial

Nada de **filho órfão** nem apagar **mãe com filhos**:

```
USER FUNCTION VALEXCSZ1()  // impede excluir contato com interações
  IF ExistCpo("SZ2", xFilial("SZ2") + SZ1->Z1_CODIGO, 1)
    MsgAlert("Contato possui interações!", "Atenção")
    RETURN .F.
  ENDIF
RETURN .T.
```

! Isto é **regra de negócio** → resolve com **IF / RETURN .F.** Validação ≠ tratamento de erro.

E se algo der errado no salvar?

Você validou o usuário. Mas e se o **banco cair**? a **rede falhar**? o **registro travar**?

Sem tratamento	Com tratamento
Trava com msg técnica	Mensagem amigável
Dados pela metade	Rollback automático
Difícil diagnosticar	Log detalhado
Usuário perdido	Sistema segue de pé

Que erro trata com o quê?

Tipo	Trata com
Validação (campo vazio)	IF / RETURN .F.
Negócio (regra violada)	IF / RETURN .F.
Runtime (divisão por zero)	BEGIN SEQUENCE
Banco de dados (lock)	BEGIN SEQUENCE
Arquivo / rede	BEGIN SEQUENCE

O **esperado** → IF. O **inesperado** → BEGIN SEQUENCE.

BEGIN SEQUENCE — o try/catch do ADVPL

```
BEGIN SEQUENCE
  nResultado := nA / nB    // divisão por zero!
RECOVER WITH oErro
  QOut("Erro: " + oErro:Description)
END SEQUENCE

QOut("Programa continua normalmente.")
```

Saída: Erro: Zero divisor → Programa continua normalmente.

O objeto oErro

Propriedade	Descrição
<code>oErro:Description</code>	O que deu errado
<code>oErro:Operation</code>	Operação que falhou
<code>oErro:ProcName</code>	Função onde ocorreu
<code>oErro:ProcLine</code>	Linha onde ocorreu
<code>oErro:SubSystem</code>	Subsistema (DBFNTX...)

Mensagem **amigável** para o usuário; esses detalhes vão para o **log**.

Salvar profissional (4 camadas)

```

BeginTran()                // 1. tudo ou nada
BEGIN SEQUENCE
  IF Empty(M->Z1_CLIENTE)    // 2. validação → IF/Break
    lok := .F. ; Break()
  ENDIF
  RecLock("SZ1", INCLUI)    // 3. lock (.T. inclui/.F. altera)
  SZ1->Z1_CODIGO := M->Z1_CODIGO
  MsUnLock()                // 4. salva e libera
RECOVER WITH oErro         // 4. inesperado:
  lok := .F. ; RollBackTran() // desfaz tudo
  MsgStop("Erro: " + oErro:Description, "Erro")
  U_GRAVARLOG("Salvar", oErro)
END SEQUENCE
IF lok ; CommitTran() ; ENDIF // 5. confirma

```

! Boas práticas (valem ouro)

1. **IF** para o esperado, **BEGIN SEQUENCE** para o inesperado
2. Toda transação faz **RollBackTran()** no erro
3. Mensagem amigável ao usuário, detalhe no log
4. Nunca engula um erro em silêncio (RECOVER vazio = bomba-relógio)

 `ErrorBlock({|oE| ...})` instala um tratador **global** — a última rede se um erro escapar de tudo.

👋 Mão na massa

Vamos codar juntos a **gravação à prova de falhas**:

```
USER FUNCTION STTIP003SALVAR()  
  LOCAL lOk := .T., oErro  
  BeginTran()  
  BEGIN SEQUENCE  
    // 1) valide Z1_CLIENTE e Z1_ASSUNTO (use Break())  
    // 2) grave com RecLock / MsUnLock  
  RECOVER WITH oErro  
    // 3) rollback + mensagem + log  
  END SEQUENCE  
  // 4) commit se lOk  
  RETURN lOk
```

No fim, cada um **provoca um erro de propósito** e vê a mensagem amigável. 🎯

Cola do Módulo 9

```
POSICIONE("SZ1",1,xFilial("SZ1")+cChave,"Z1_NOME") // busca em outra tabela
ExistCpo("SZ1",xFilial("SZ1")+cChave,1)           // .T. se a chave existe
PRIVATE aRotina := { {"Interações","U_STTIP004",0,6} } // botão customizado

BeginTran() / CommitTran() / RollBackTran()        // transação: tudo ou nada
RecLock("SZ1",.T.) ... MsUnLock()                  // .T. inclui .F. altera
BEGIN SEQUENCE ... RECOVER WITH oErro ... END SEQUENCE // try/catch do ADVPL
```

Exercícios (entrega até 03/08)

Obrigatórios (o essencial, roda até em Harbour puro):

5. Primeiro contato com erros (`BEGIN SEQUENCE`)

6. Gravação à prova de falhas (transação + lock + log)

Bônus (sem nota, material de autoestudo): 1 dicionário · 2 lib e rotinas · 3 gatilhos e validações · 4 menu · 7 integridade/executor genérico.

Detalhes no `exercicios.md` — um arquivo por exercício. Suporte no Discord até **07/08**. 🎉

➡ Agora: o TCC 🎓

Fechamos a trilha! O que vem agora é o seu **projeto final**:

- Um sistema real de **não conformidades ISO 9001** — evolução direta do que fizemos hoje
- Entrega no **GitHub**, validável por IA · **prazo 07/08**
- Tem **núcleo mínimo** alcançável e um teto para quem quer brilhar

Tudo que você construiu na jornada converge aqui. Bora fechar com chave de ouro. 🏁

Bora blindar o sistema!

Dúvidas na trilha? **Discord** [#duvidas-parte2-protheus](#)

JORNADA DEV START